

Software Models and UML

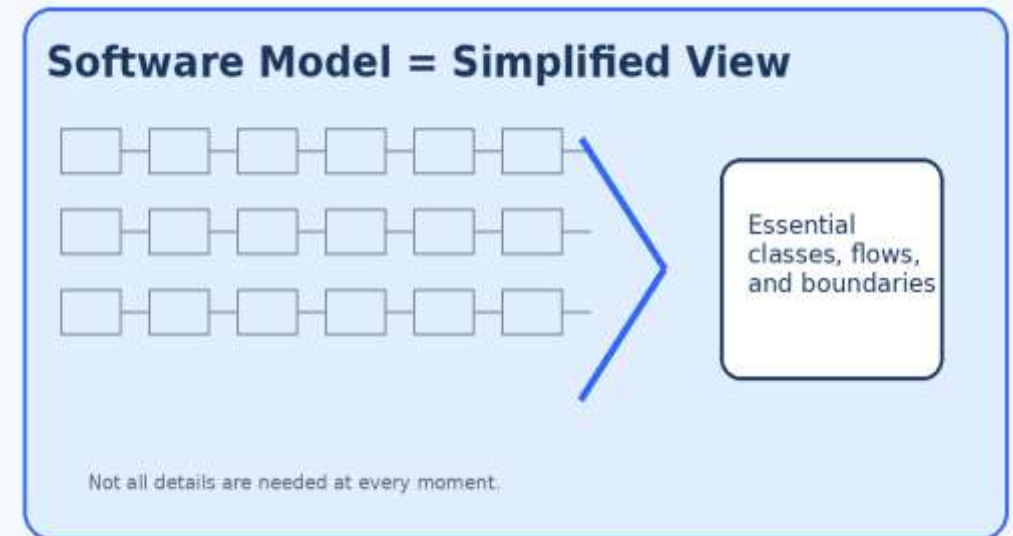
CSC 4110 : Prepared by Arif Shariar Rahman

Learning objectives

- Explain why software engineers use models.
- Differentiate predictive, extracted, and prescriptive models.
- Identify major UML diagram families: structure and behavior diagrams.
- Draw basic activity, use case, and class diagrams.
- Use actors, system boundaries, include/extend/generalization relationships, multiplicity, aggregation, composition, and dependency correctly.
- Write a structured use case description using preconditions, basic flow, alternatives, postconditions, and special requirements.

Why do we need software models?

- A model is a simplified view of a software system.
- It retains the important properties and omits unnecessary clutter.
- Different stakeholders need different views:
 - Customers care about user goals and workflows.
 - Developers care about classes, interfaces, dependencies, and data flow.
 - Project managers care about risk, scope, and effort.
- Good models help teams reason about complexity before mistakes become expensive.



Roles of software models

Model role	When created	Main purpose	Example
Predictive / Design	Before implementation	Predict structure, behavior, risk, and effort	Class diagram before coding a hospital system
Extracted	After or during implementation	Answer questions about existing code	Reverse-engineered dependency diagram
Prescriptive	Maintained with code	Define rules/constraints for evolution	Architecture rule: UI cannot access database directly

Key point: “model” does not always mean “up-front documentation”; it can also explain or govern existing software.

Criteria for successful software modeling

Essential information retained
Classes, actors, flows, rules, dependencies

Clutter removed
Low-level implementation details hidden when they are not needed

Audience-aware
Model speaks to the stakeholder's current question

Consistent with system
Model does not contradict the code or requirements

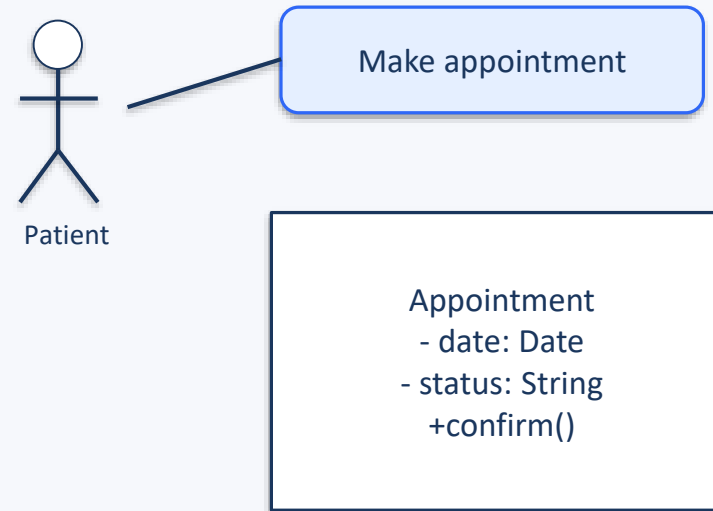
Warning: a model that omits important information can become misleading and cause design mistakes.

What is UML?

Unified Modeling Language: a standard graphical notation for software analysis and design.

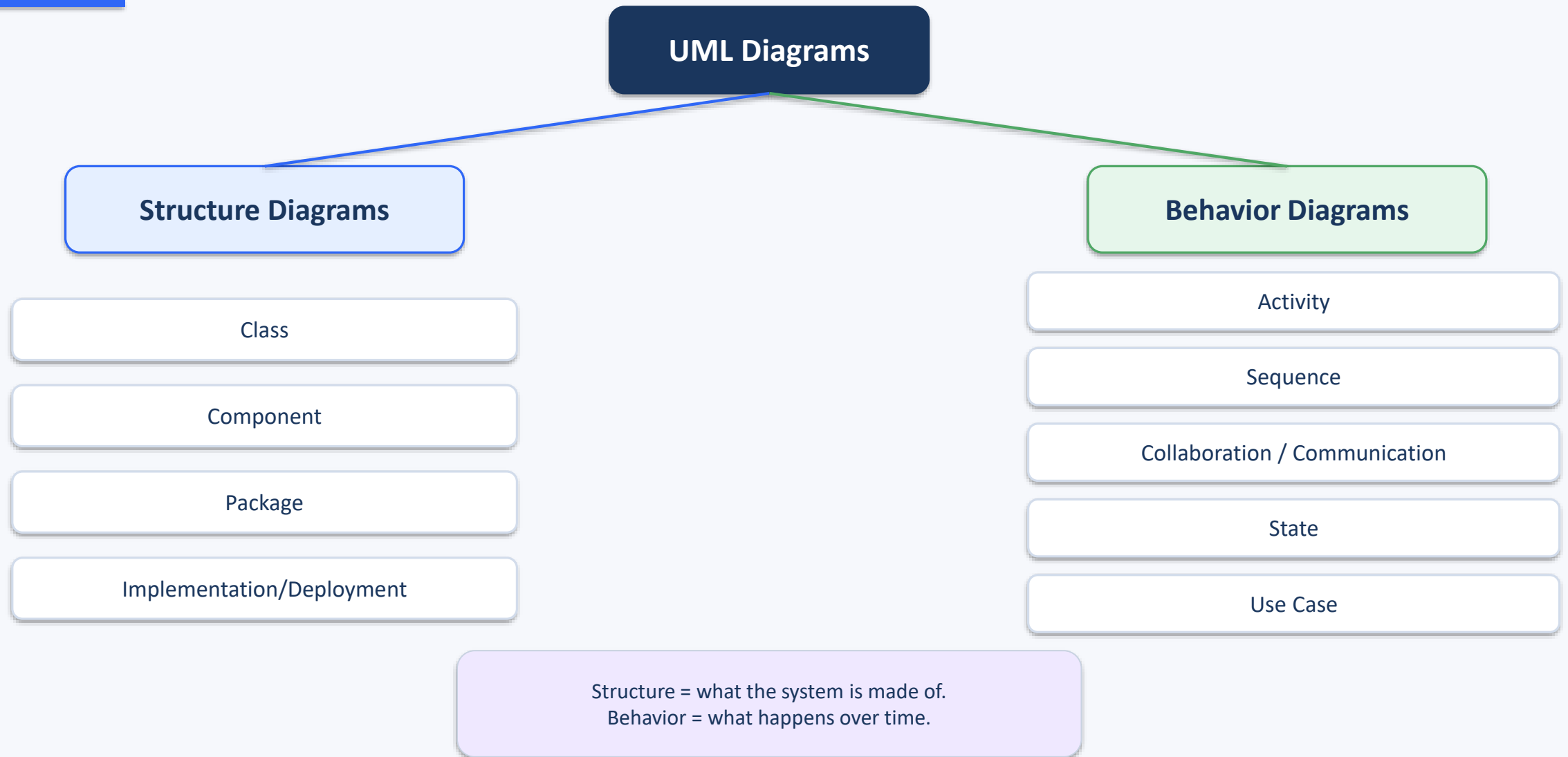
- UML provides pictures or views of an object-oriented system.
- It helps communicate system structure and behavior before or during implementation.
- It is not a programming language; it is a modeling language.
- It is useful because “a picture is worth a thousand words.”

OO System: Hospital Appointment App



UML is most effective when every diagram answers a specific question.

UML diagram families

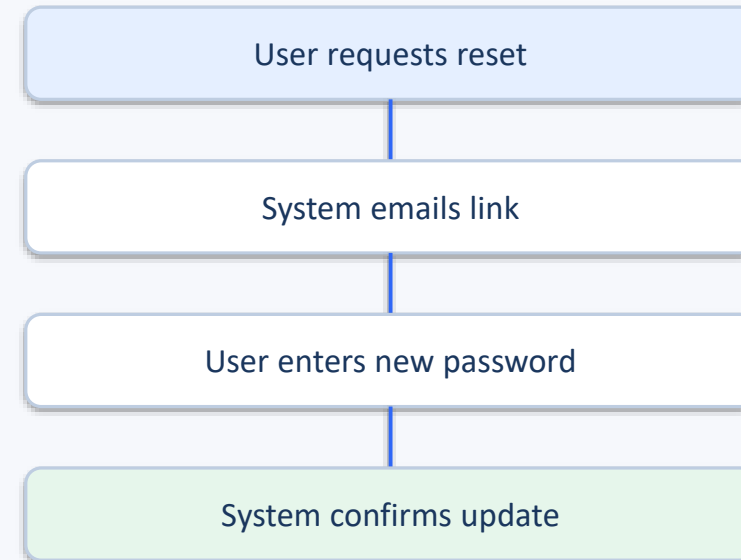


Activity diagrams: modeling workflows

Activity diagrams are behavior diagrams: they describe what must happen.

- Demonstrate the logic of an algorithm.
- Describe the steps performed in a use case.
- Illustrate a business process or workflow between users and the system.
- Clarify complicated flows and alternative paths.
- Model operations, functions, or method-level behavior.

Example: Password reset workflow



Components of an activity diagram

Each symbol has a precise meaning.



**Start node
black circle**

Action

**Action
rounded rectangle**



**Decision node
diamond with guard
conditions**



**Control flow
connector showing order**



**End node
outlined final circle**

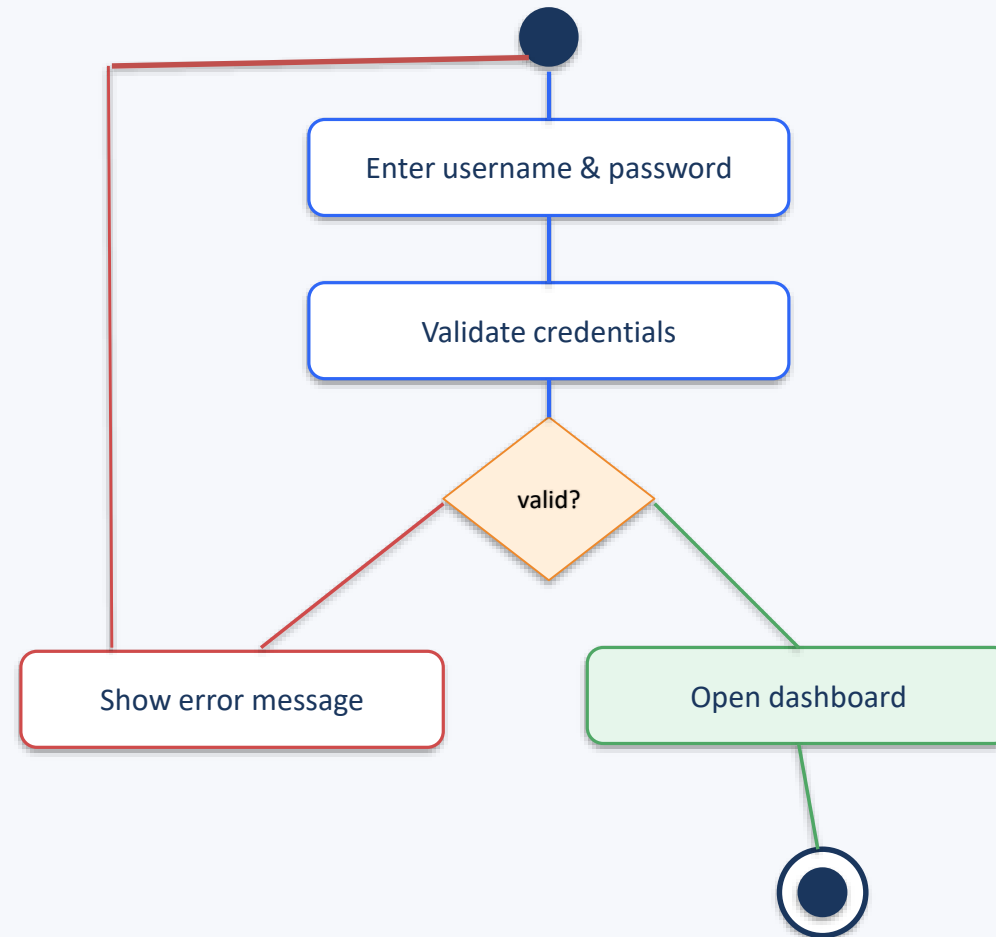
Common mistake: using a decision node without labeling the outgoing conditions, such as [valid] and [invalid].

Activity diagram example: User login

This expands the simple “user login” example into an executable workflow.

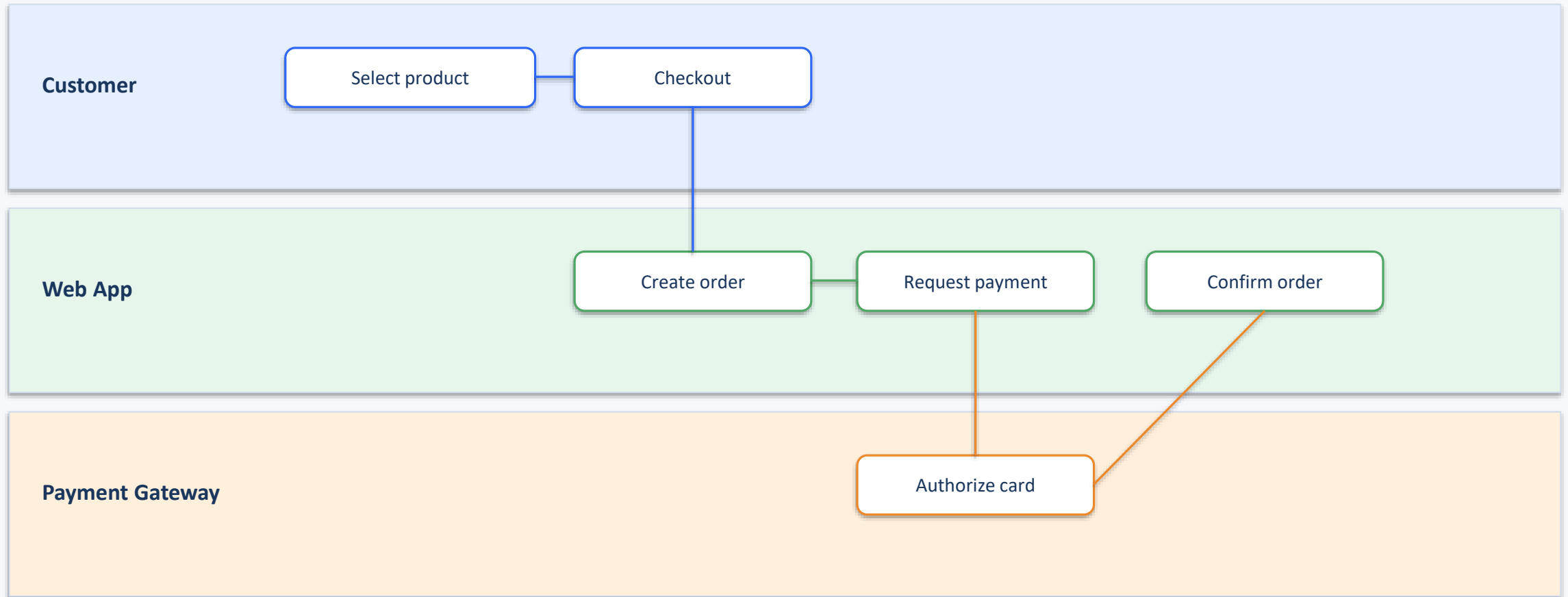
Scenario notes

- Normal path: credentials are valid, user sees dashboard.
- Alternative path: invalid credentials, system displays error.
- Guard conditions should be explicit: [valid], [invalid], [retry].



Swim lanes: who performs each action?

Swim lanes partition an activity diagram by responsibility.

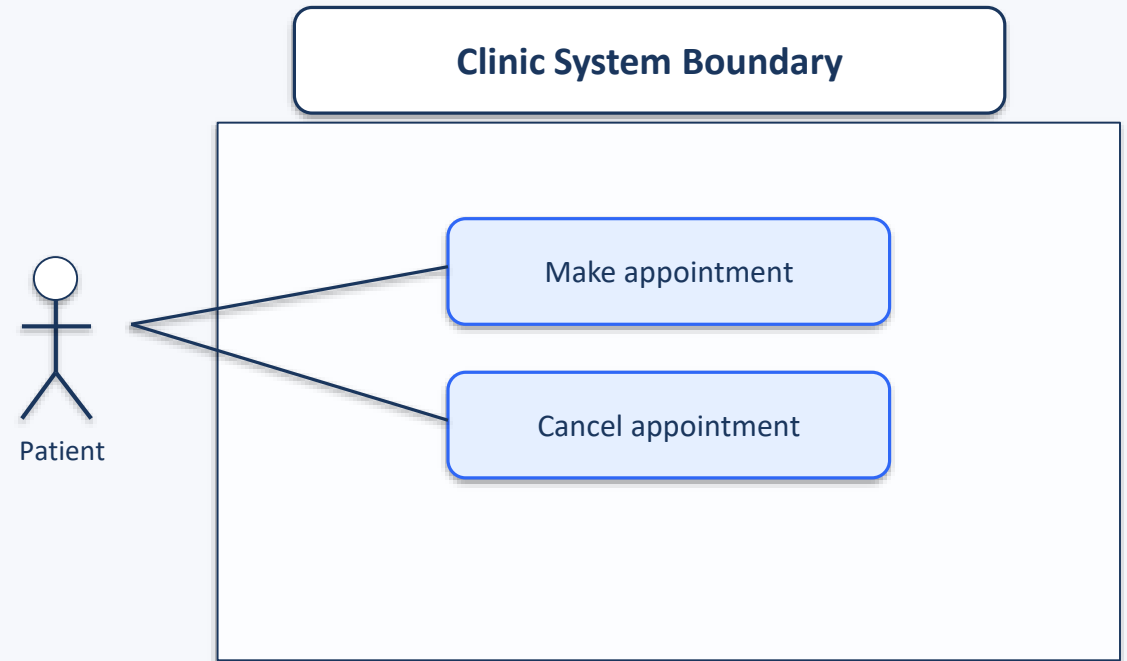


Swim lanes make responsibility explicit: user action vs system action vs external service action.

Use case diagrams: external view of system behavior

Use case diagrams emphasize what a system does, not how it does it.

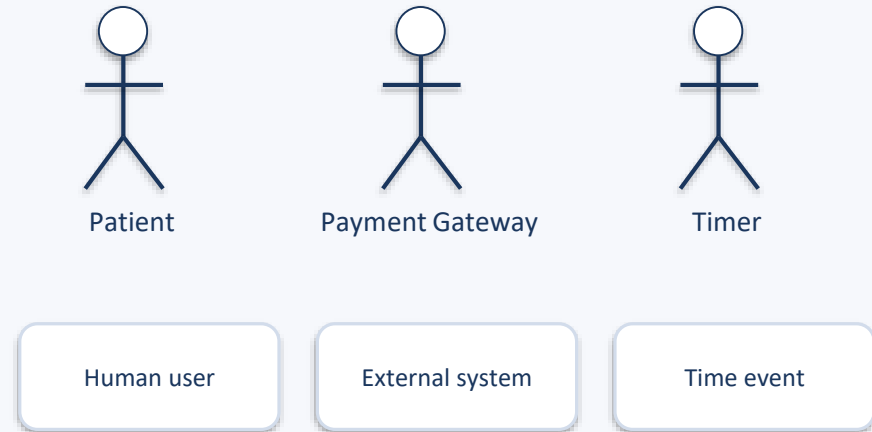
- A use case describes a goal-oriented interaction between an actor and a system.
- It is closely connected to scenarios: examples of what happens when someone interacts with the system.
- Use cases help discover requirements and system scope.
- They are most useful early in requirements analysis and stakeholder communication.



Actors in use case diagrams

An actor is outside the system and interacts with it.

- Actors may be human users, peripheral devices, external systems, subsystems, or time-based events.
- Actors are represented with stick figures and must be labeled.
- A single actor can represent many physical users.
- Actors are not part of the system boundary.



Ask: “Does this thing initiate or participate in a scenario?” If yes, it may be an actor.

Actor vs stakeholder

A stakeholder may care about the system without directly acting in a scenario.

Actor

Has behavior that interacts with the system.

Primary actor initiates an interaction to achieve a goal.

Supporting actor helps the use case complete a sub-goal.

Stakeholder

Has an interest in the system outcome.

May not interact with the system in any scenario.

Examples: supplier, vendor, stock agency, regulator, manager.

Example: A hospital administrator is a stakeholder; they become an actor only if they log in and use administrative functions.

How to identify actors

Use these questions during requirements elicitation.

Who is interested in the scenario or system?

Where in the organization will the system be used?

Who benefits from the system?

Who supplies, uses, or removes information?

Does one person play several different roles?

Do several people play the same role?

Classroom prompt: identify actors for a Wayne State course registration system.

Use cases and scenarios

Each use case represents one user-visible function or goal.

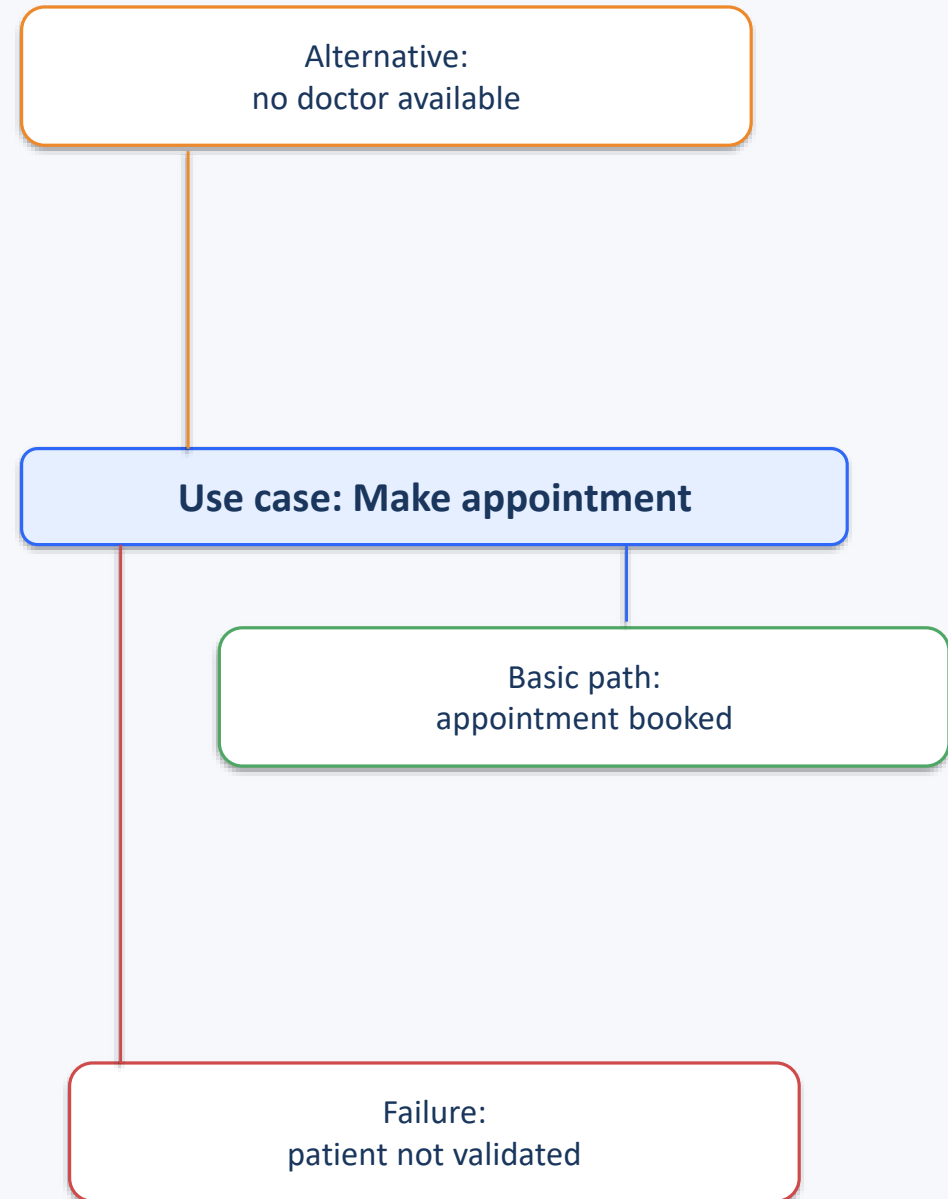
A use case can contain several paths through the system.

Each path is a scenario.

Use cases are drawn as ovals/ellipses.

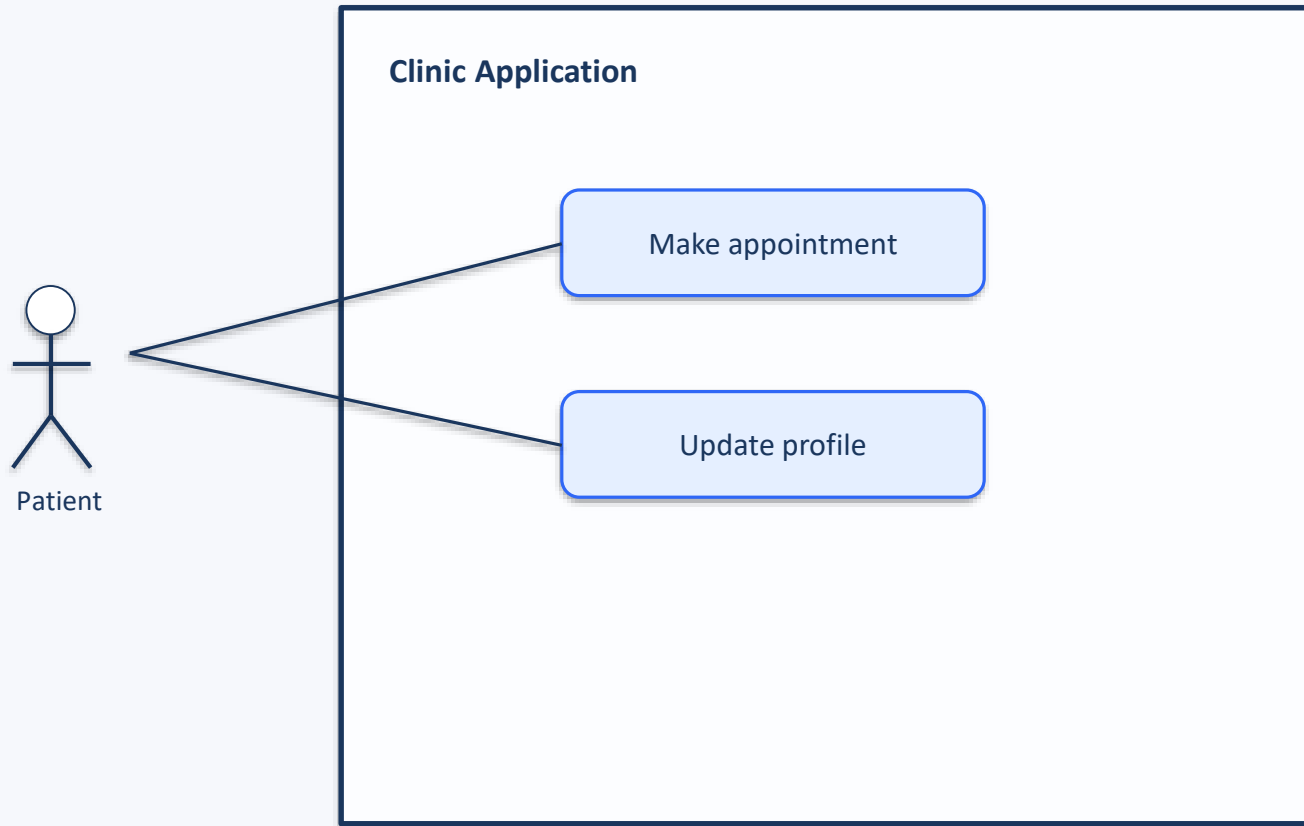
Actors are stick figures; communications are lines connecting actors to use cases.

Use case names should be active verb phrases:
“Make appointment,” “Withdraw cash,” “Generate report.”



Use case relationships and system boundary

A boundary rectangle defines the scope of the system.



Association: actor communicates with use case.

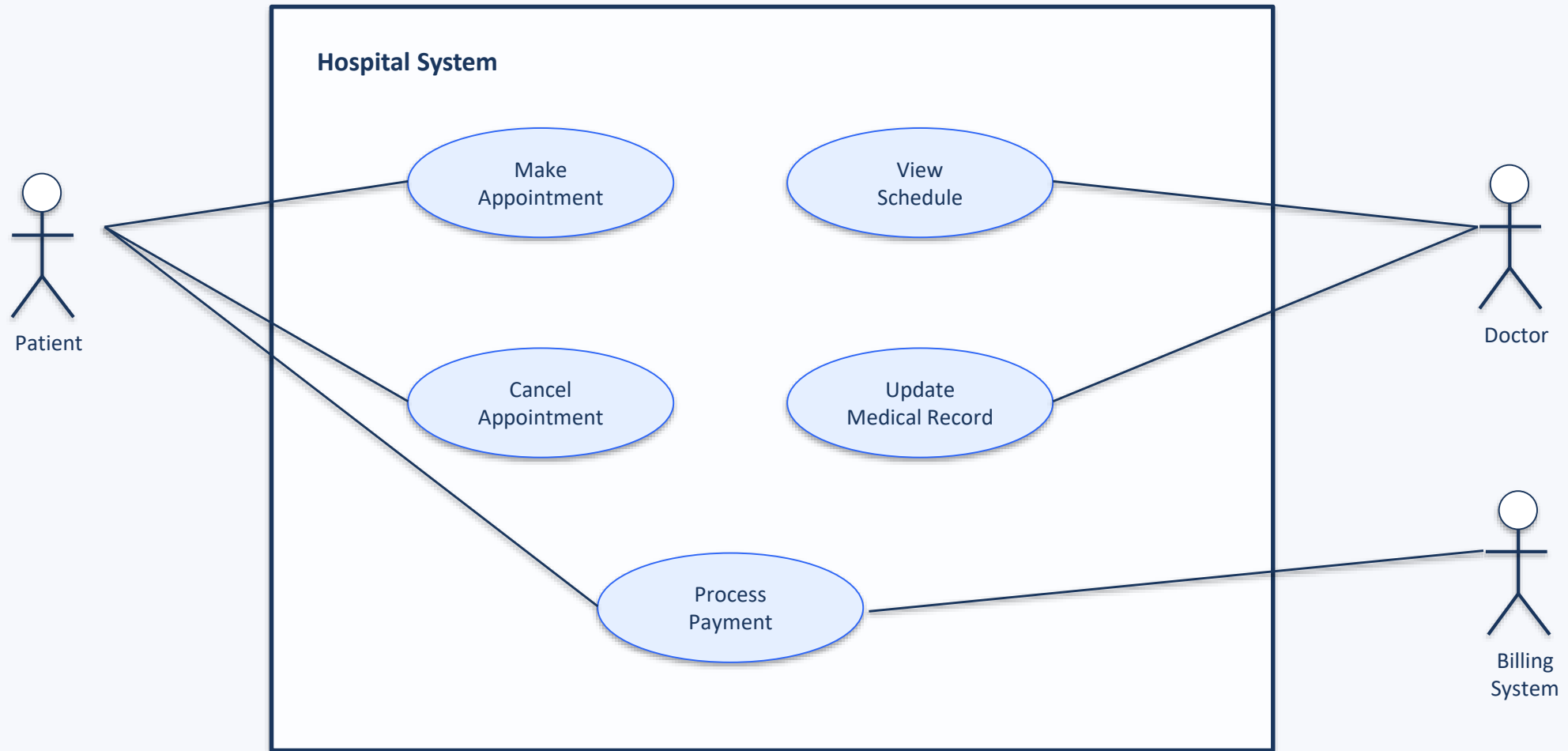
Boundary: what is inside the system scope.

Actors remain outside the rectangle.

Use cases inside the boundary must be functions provided by the system.

Sample use case diagram: Hospital / clinic system

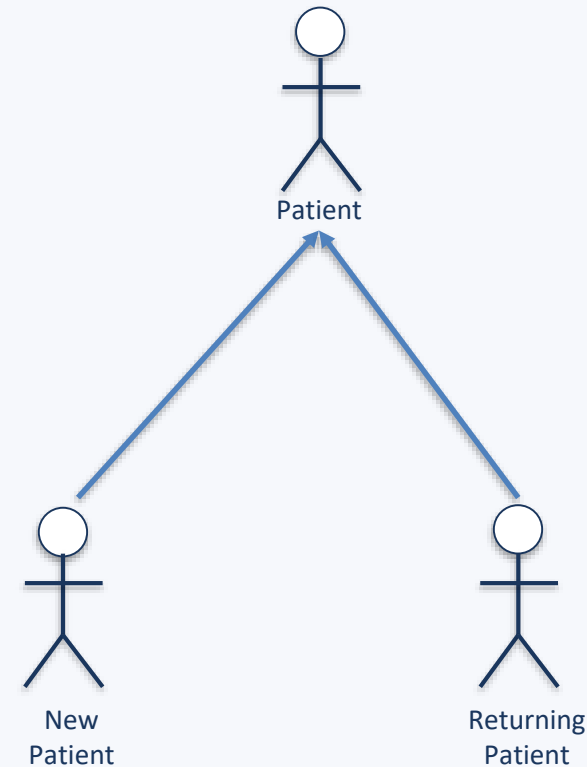
A realistic example that combines multiple actors and use cases.



Use case relationship: Generalization

Generalization represents inheritance from child to parent.

- Use when specialized actors or use cases share common behavior.
- Arrow points from child to parent.
- It is represented by a line with a hollow triangle arrowhead.
- Example: New Patient and Returning Patient are specialized forms of Patient.



In UML notation, the hollow triangle points to the more general parent.

Use case relationships: <<include>> and <<extend>>

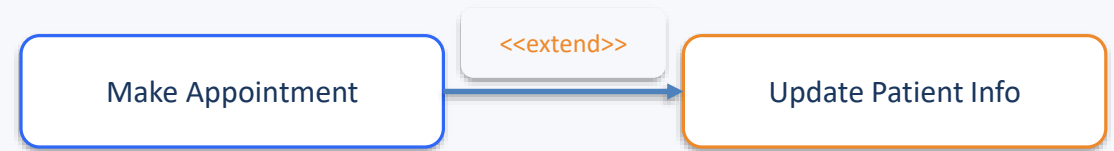
These two relationships are often confused.

<<include>> = required reusable behavior



The base use case always uses the included use case.
Good for common required functionality such as authentication.

<<extend>> = optional or conditional behavior



The extension happens only under a condition.
Example: update patient info only if profile is outdated.

Use case relationship summary

Choose the relationship based on meaning, not visual convenience.

Relationship	Meaning	Arrow direction	Example
Association	Actor communicates with use case	Actor ↔ use case	Patient — Make appointment
Generalization	Child specializes parent	Child → Parent	New Patient → Patient
<<include>>	Required reused behavior	Base → Included	Withdraw Cash → Validate User
<<extend>>	Optional/conditional behavior	Extension → Base	Update Info → Make Appointment

Rule of thumb: include = mandatory subfunction; extend = conditional addition.

Use case textual template

A diagram shows scope; a use case description gives operational detail.

Brief description
What task is being performed?

Actors
Primary and secondary actors

Preconditions
What must already be true?

Basic flow
Successful scenario steps

Alternative flows
What if something goes wrong?

Postconditions
What is true after success/failure?

Special requirements
Rules, constraints, guidelines

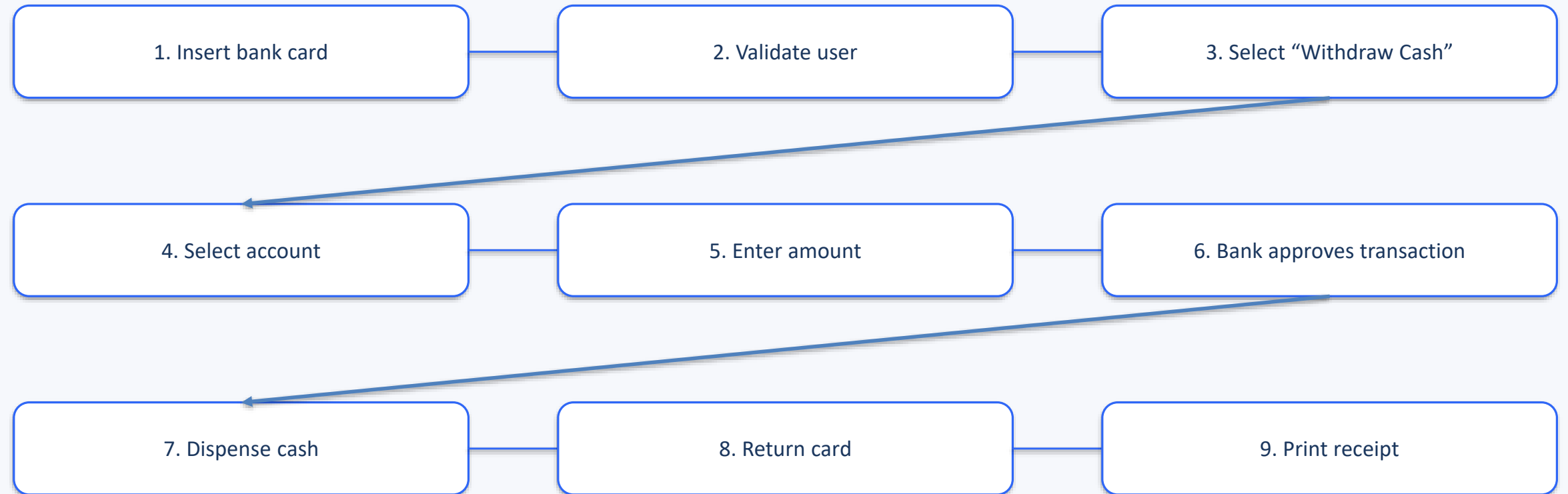
Example use case: ATM Withdrawal

A formal use case turns a diagram into testable requirements.

ID	FR1: ATM_Withdraw
Description	Bank customer withdraws money from a bank account using an ATM.
Actors	Primary: Bank Customer Secondary/supporting: Bank, ATM Machine
Preconditions	1. Active network connection to the bank. 2. ATM has cash available.

ATM Withdrawal: basic successful flow

A basic flow describes the normal success path.



Each step should be observable enough to support review, implementation, or test design.

ATM Withdrawal: alternative flows and constraints

Alternative flows make the requirements realistic and testable.

Invalid user

Validate User fails → use case ends with failure condition.

Wrong account

Bank card is not associated with selected account → show message and return to account selection.

Wrong amount

Amount cannot be created with available bills → ask customer to reenter amount.

Exceeds individual limit

Amount exceeds maximum withdrawal → show warning and return to amount entry.

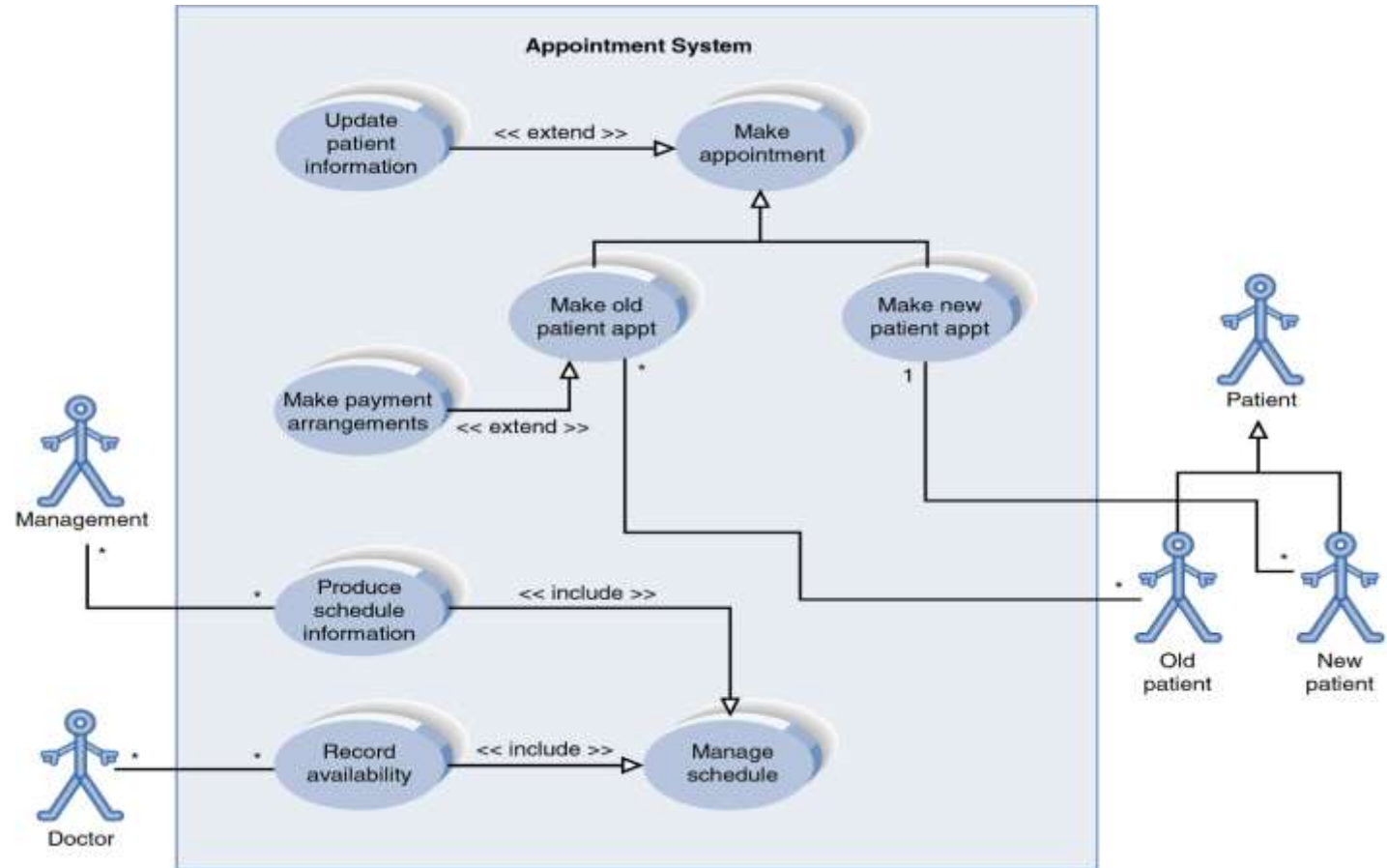
Exceeds daily limit

Bank rejects transaction due to daily limit → show warning and return to amount entry.

Special requirements

Cash in multiples of \$20; maximum individual withdrawal \$500; log complete and incomplete transactions.

Appointment System



UML class diagrams

A class diagram shows the static structure of an object-oriented system.

- A class diagram pictures classes in an OO system.
- It shows fields/attributes and methods/operations.
- It shows connections between classes that interact or inherit from one another.
- It does not show algorithmic details or the exact sequence of object interactions.

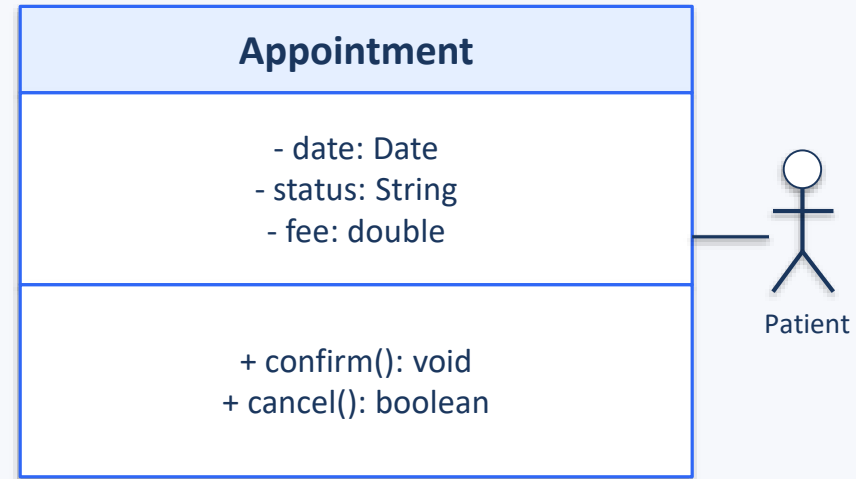
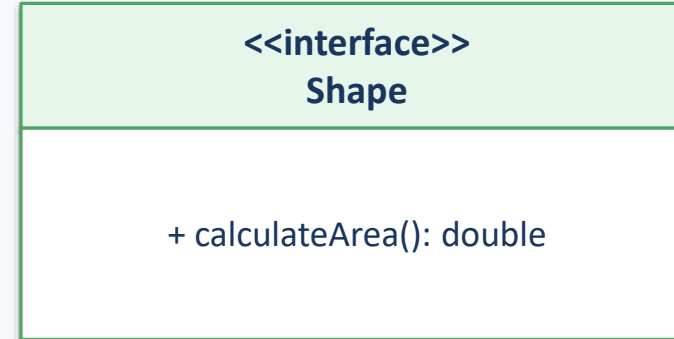
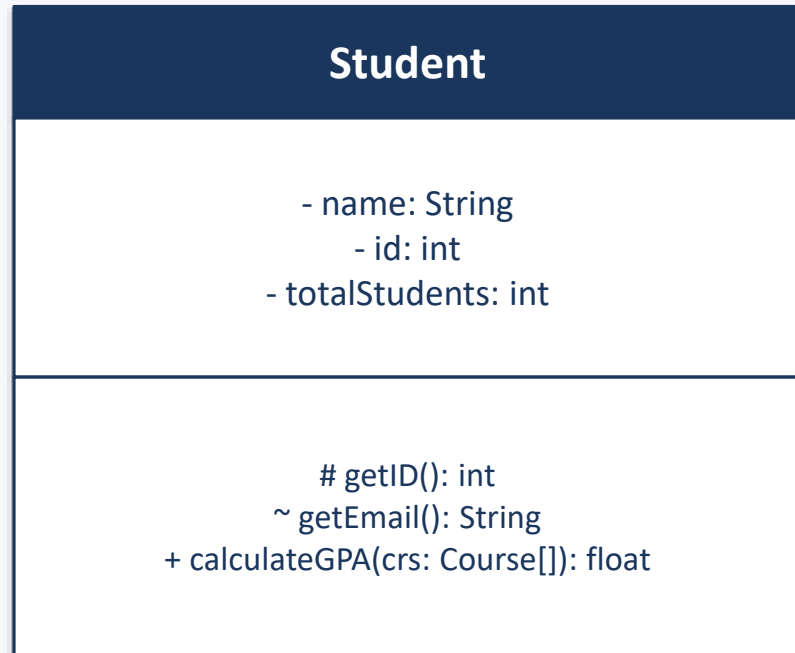


Diagram of a single class

A class rectangle is divided into name, attributes, and operations.



- Class name goes on top. Attributes are optional in the middle. Operations are optional at the bottom.
- For interfaces, do not omit the methods that define the interface contract.

Class attributes and operations

Visibility tells who can access a field or method.

Symbol	Visibility	Example
+	public	+ name: String
#	protected	# age: int
-	private	- DOB: String
~	package/default	~ courses[100]: Course

Operation signature pattern

visibility name(parameters) : return_type

+ calculateGPA(crs: Course[]) : float
+ Student(n: String, dob: String)

Constructors normally omit return type; void methods may omit return type in many UML teaching contexts.

Relationships between classes

Class diagrams become powerful when relationships are modeled correctly.

Generalization
Inheritance or interface implementation

Association
Objects are connected or use one another

Aggregation
Weak “part-of” relationship

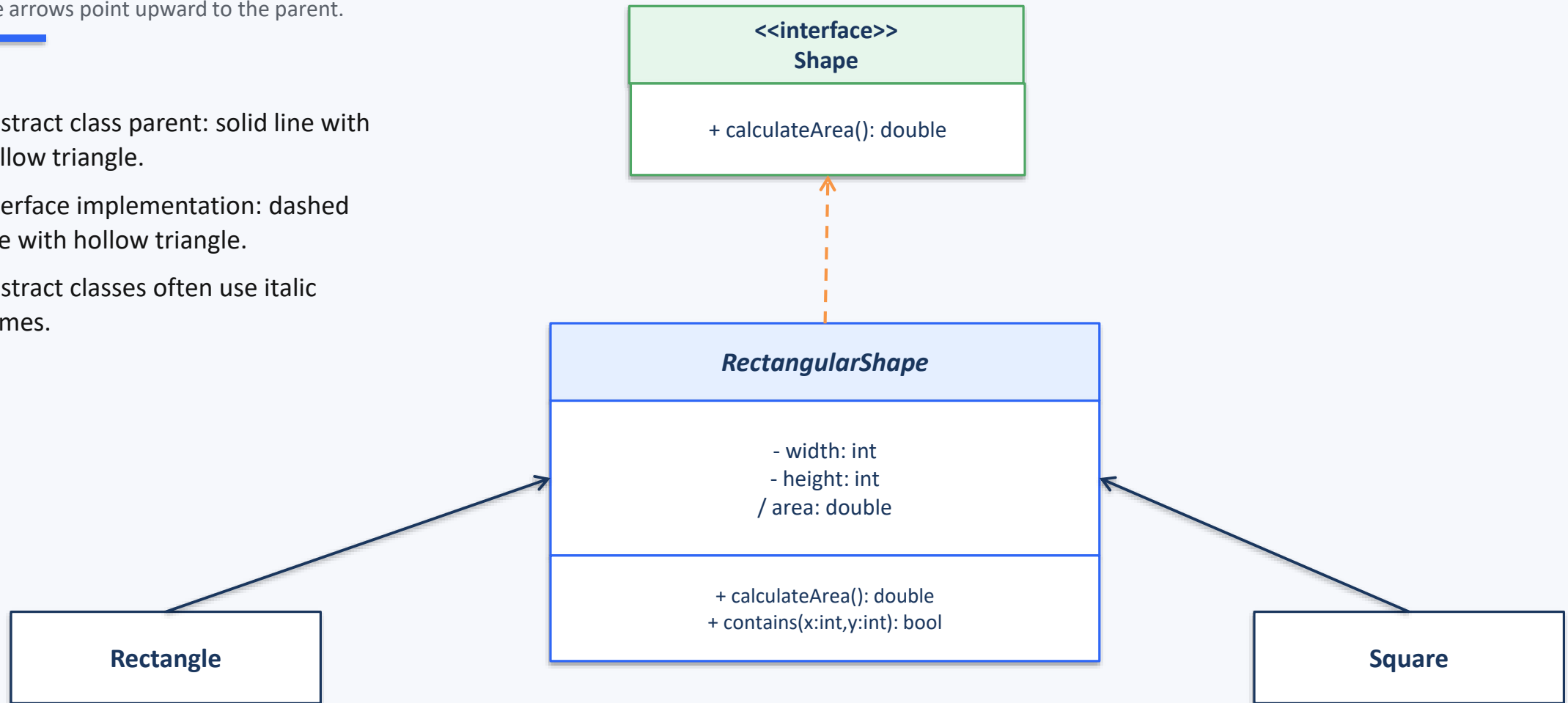
Composition
Strong whole-part lifetime ownership

Dependency
Temporary use; often implementation detail

Class relationship: Generalization

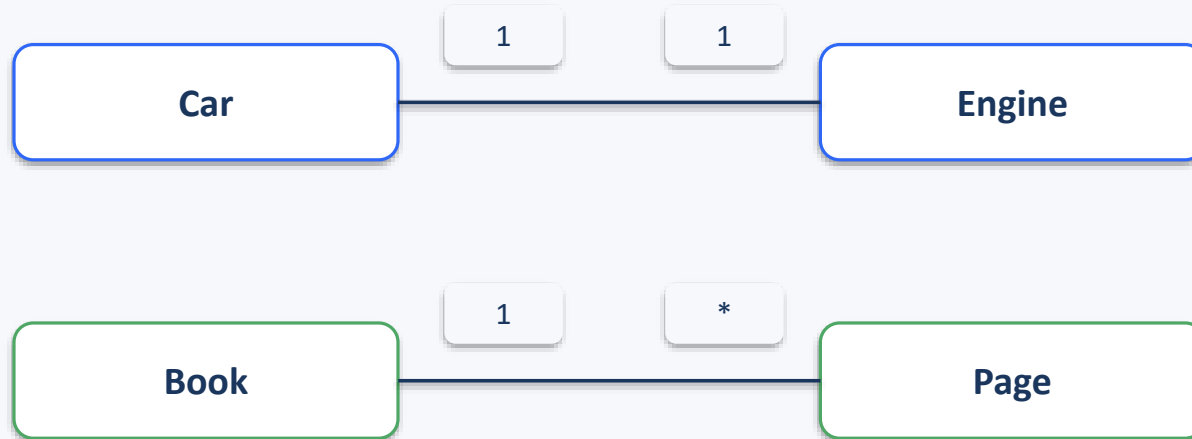
Inheritance arrows point upward to the parent.

- Abstract class parent: solid line with hollow triangle.
- Interface implementation: dashed line with hollow triangle.
- Abstract classes often use italic names.



Association multiplicity

Multiplicity specifies how many objects participate in a relationship.



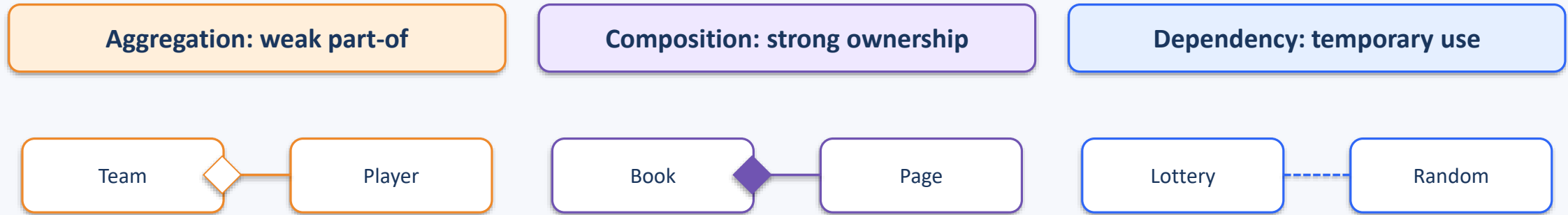
Common multiplicities

1 = exactly one
* = zero or more
2..4 = between 2 and 4
3..* = three or more
0..1 = optional / at most one

Multiplicity is one of the most testable pieces of a class diagram.

Association types: aggregation, composition, dependency

The diamond and line style change the meaning.



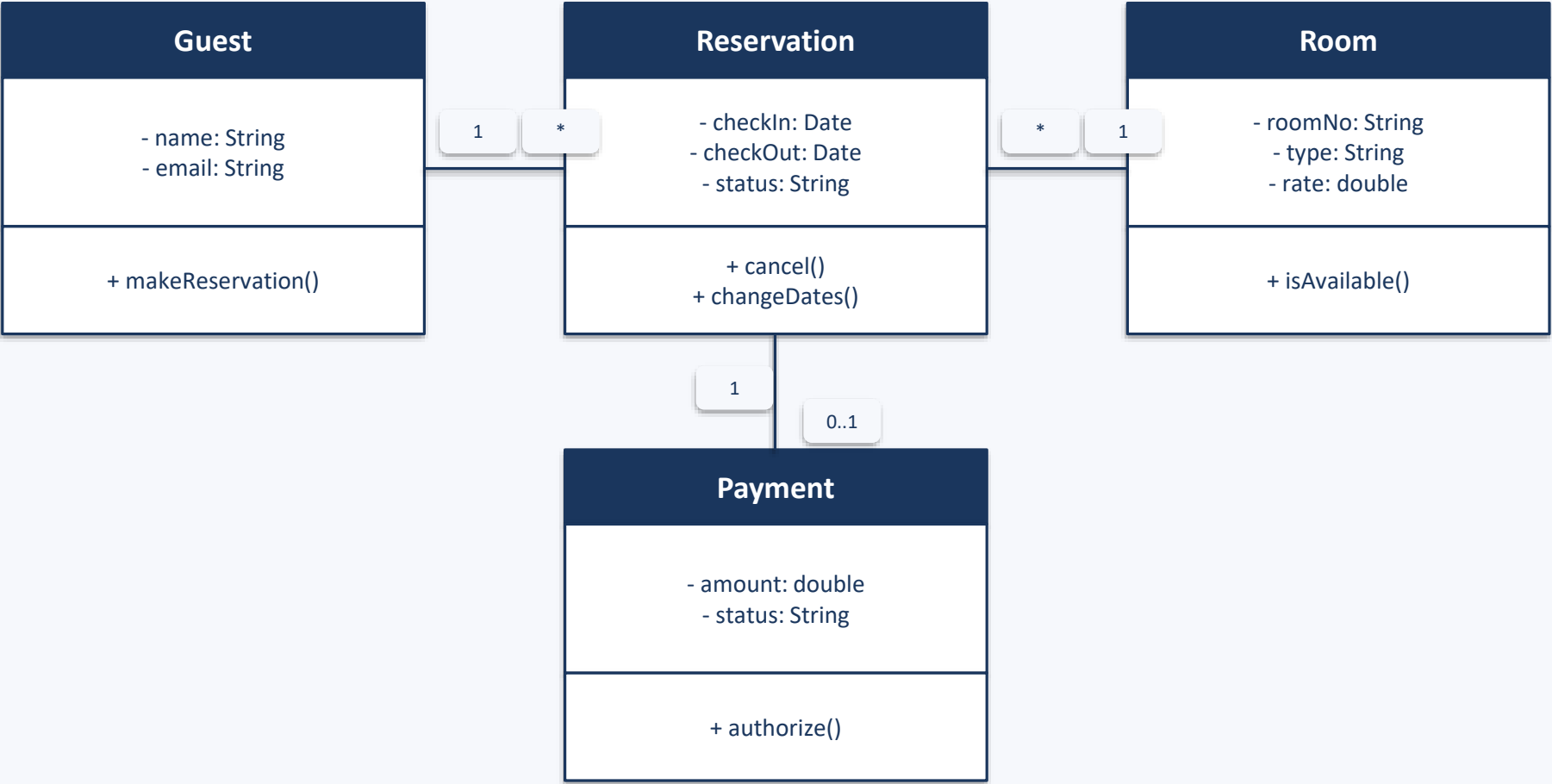
Aggregation: the part can exist independently of the whole.

Composition: the parts live and die with the whole.

Dependency: one class uses another temporarily; often not an intrinsic part of object state.

Class diagram example: Hotel reservation system

This example connects the UML class concepts to a real project domain.



Interpretation

A guest can have many reservations. Each reservation books one room and may have one payment.

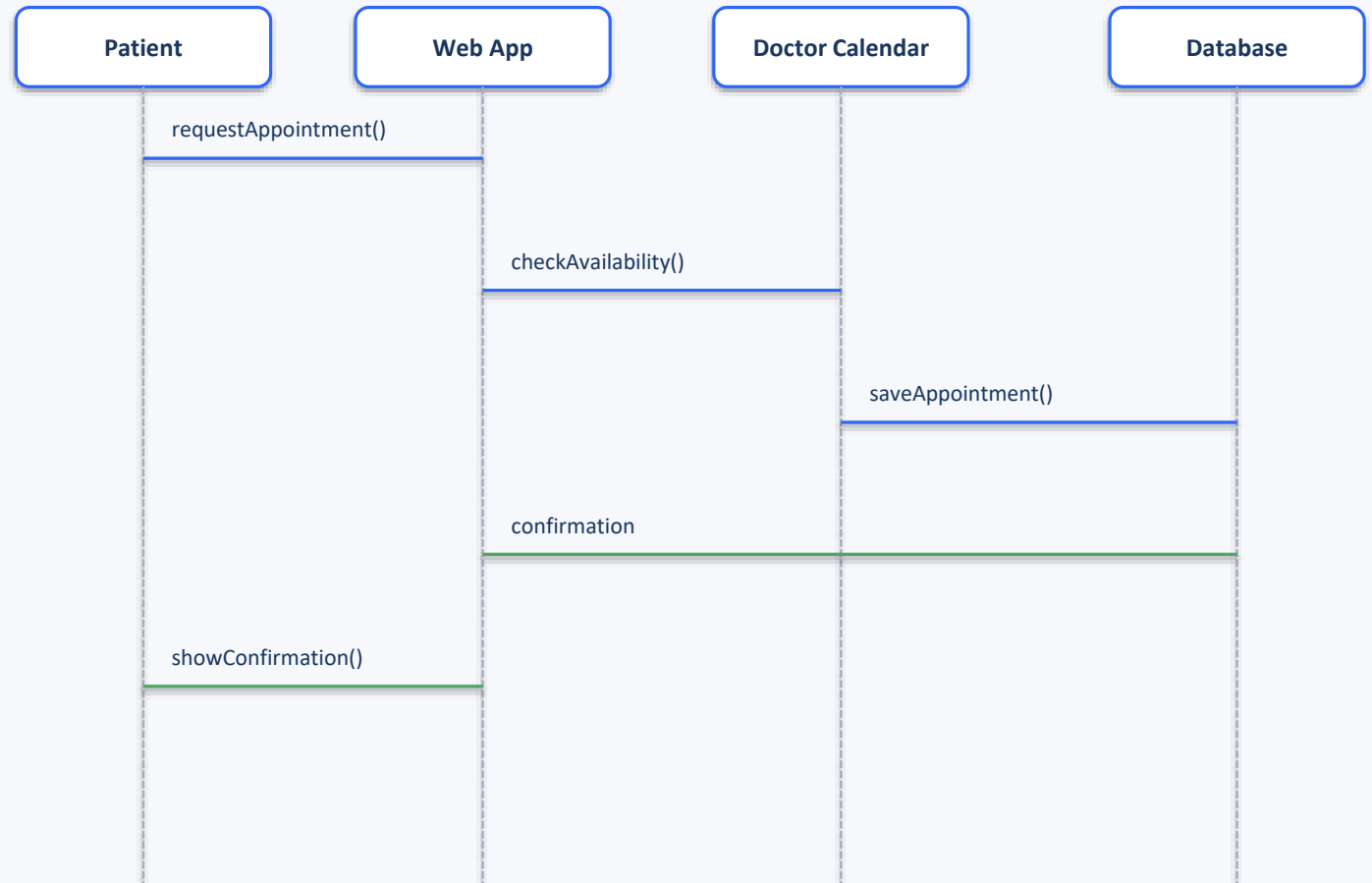
Sequence diagrams: time-ordered interaction view

A sequence diagram complements a class diagram by showing messages over time.

Shows objects or actors across the top and time flowing downward.

Useful when the question is “who sends what message to whom, and in what order?”

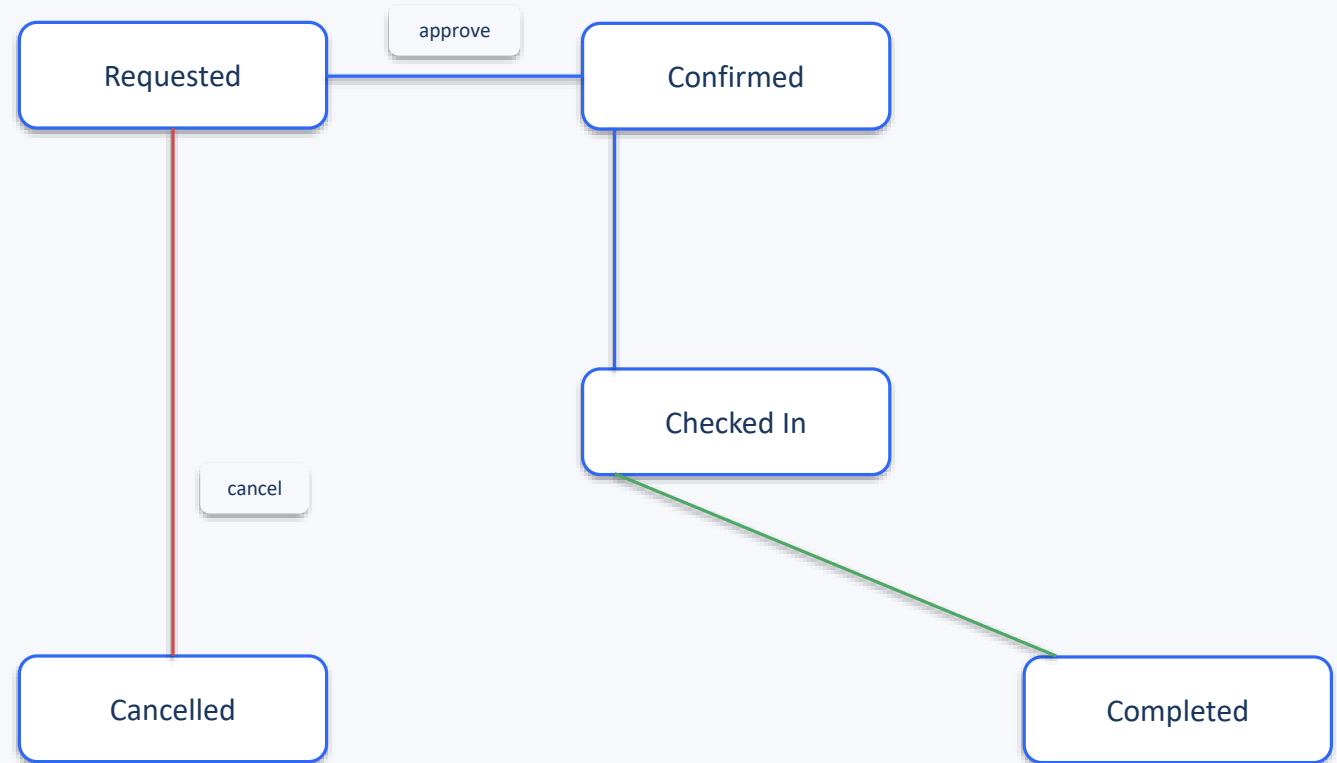
Example: booking an appointment involves Patient, Web App, Doctor Calendar, and Database.



State diagrams: lifecycle of one object

A state diagram is useful when an object has important status changes.

- Shows valid states of an object and transitions between them.
- Useful for objects such as Reservation, Order, Ticket, Account, or Payment.
- Complements activity diagrams: activity diagrams show workflow; state diagrams show object lifecycle.



Common UML mistakes to avoid

UML is valuable only when notation and meaning match.

Putting actors inside system boundary

Fix: Actors are external to the system.

Using include/extend randomly

Fix: Include is required; extend is optional/conditional.

Drawing every implementation detail

Fix: A model should simplify, not duplicate code.

Omitting multiplicity

Fix: Multiplicity often encodes key requirements.

Treating class diagrams as database ERDs

Fix: They overlap, but class diagrams model objects and behavior too.

Leaving diagrams stale

Fix: A misleading model is worse than no model.